네온 보드게임

Technical Documentation

Real-time Multiplayer Board Game Platform
React + Node.js + Socket.IO + Docker

목차

1. 프로젝트 개요
2. 기술 스택
3. 시스템 아키텍처
4. 디렉토리 구조
5. 인증 플로우
6. 실시간 통신 설계
7. 게임 로직 설계
8. 클라이언트 구조
9. 스타일링 및 테마
10. Docker 배포
11. 서버 상태 관리
12. 보안 고려사항
13. 확장 가이드

1. 프로젝트 개요

웹 기반 실시간 멀티플레이어 보드게임 플랫폼이다. 닉네임으로 간편하게 입장하여 방을 만들고, 다른 유저와 보드게임을 즐길 수 있다. 관전, 채팅, 테마 전환 등 부가 기능을 포함한다.

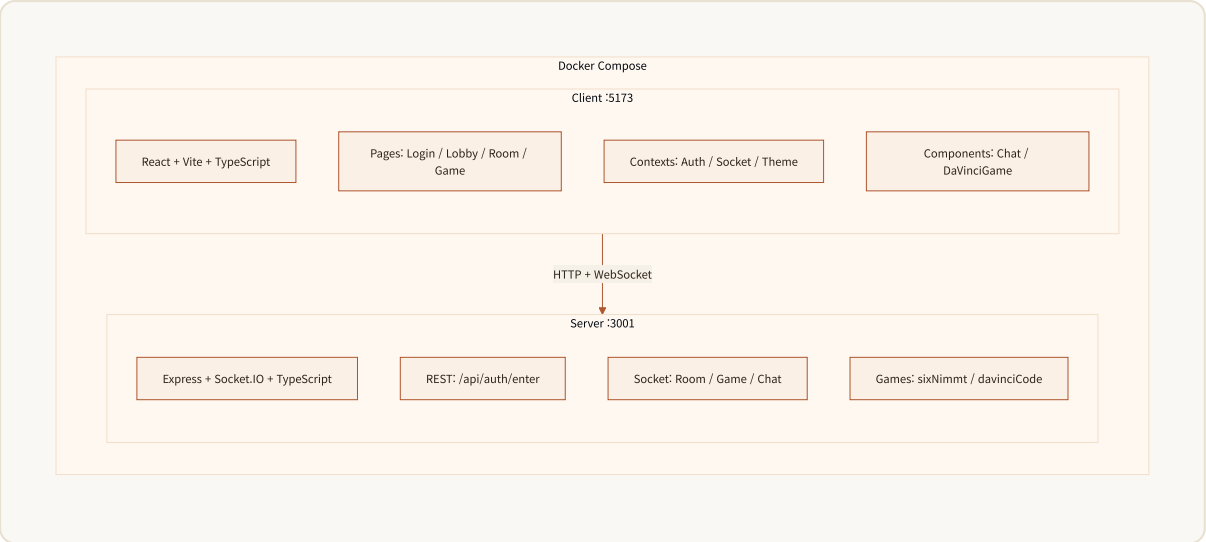
지원 게임

게임	인원	방식
젝스님트 (6 Nimmt!)	2~10명	동시 카드 선택, 열 배치, 벌점 누적
다빈치 코드 (Da Vinci Code)	2~4명	순차 타일 추측, 추리, 탈락제

2. 기술 스택

레이어	기술	버전	역할
Frontend	React + TypeScript	18.2 / 5.3	UI 렌더링, 상태 관리
빌드 도구	Vite	5.0	개발 서버, HMR, 번들링
Backend	Node.js + Express	20 / 4.18	API 서버, 정적 라우팅
실시간 통신	Socket.IO	4.7	양방향 이벤트 기반 통신
인증	JWT	9.0	토큰 기반 인증
컨테이너	Docker Compose	-	서비스 오케스트레이션
폰트	Google Fonts	-	Noto Serif KR, Source Serif 4

3. 시스템 아키텍처

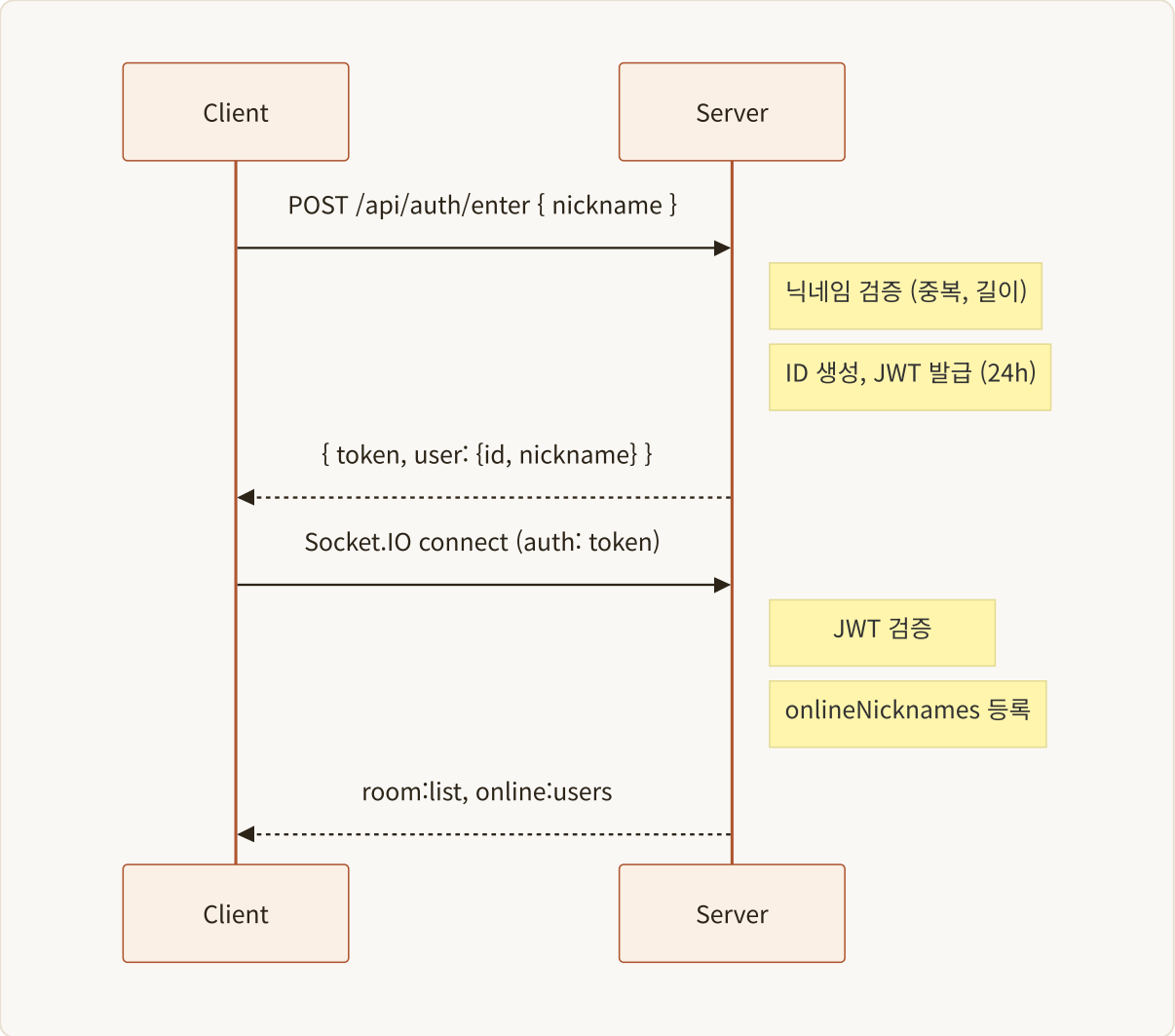

Docker Compose
Client :5173
React + Vite + TypeScript
Pages: Login / Lobby / Room / Game
Contexts: Auth / Socket / Theme
Components: Chat / DaVinciGame
HTTP + WebSocket
Server :3001
Express + Socket.IO + TypeScript
REST: /api/auth/enter
Socket: Room / Game / Chat
Games: sixNimmt / davinciCode

4. 디렉토리 구조

```
ProjectBoard/
├── docker-compose.yml          서비스 오케스트레이션
├── start.bat / stop.bat        실행/종료 스크립트
│
├── server/
│   ├── Dockerfile
│   └── src/
│       ├── index.ts            Express + Socket.IO 초기화
│       ├── routes/auth.ts      인증 엔드포인트
│       ├── middleware/auth.ts  JWT 미들웨어
│       ├── socket/index.ts     소켓 이벤트 핸들러 (방, 게임, 채팅)
│       └── games/
│           ├── sixNimmt/logic.ts     젝스님트 로직
│           └── davinciCode/logic.ts  다빈치 코드 로직
│
├── client/
│   ├── Dockerfile
│   ├── vite.config.ts
│   ├── index.html
│   └── src/
│       ├── App.tsx             라우팅 및 인증 가드
│       ├── contexts/
│       │   ├── AuthContext.tsx   인증 상태
│       │   ├── SocketContext.tsx 소켓 연결
│       │   └── ThemeContext.tsx  테마 전환
│       ├── pages/
│       │   ├── Login.tsx       닉네임 입력
│       │   ├── Lobby.tsx       로비
│       │   ├── Room.tsx        대기실
│       │   └── Game.tsx        게임 (젝스님트 + 분기)
│       ├── components/
│       │   ├── Chat.tsx        채팅 UI
│       │   ├── ChatPanel.tsx   채팅 패널 (PC/모바일)
│       │   └── DaVinciGame.tsx 다빈치 코드 UI
│       └── styles/index.css    전역 스타일, 테마, 반응형
│
└── docs/
    ├── TECHNICAL.md            기술 문서 (Markdown)
    └── TECHNICAL.html          기술 문서 (본 파일)
```

5. 인증 플로우

데이터베이스 없이 닉네임 기반 간편 인증을 사용한다. 서버 메모리에서 접속 중 닉네임을 관리하여 중복을 방지한다.



- **닉네임 중복 검사**: `onlineNicknames` Set으로 관리, 접속 해제 시 자동 삭제
- **토큰 저장**: 클라이언트 localStorage
- **소켓 인증**: Socket.IO handshake의 `auth` 필드에 JWT 포함

6. 실시간 통신 설계

모든 게임 로직과 방 관리는 Socket.IO 이벤트로 처리된다. 서버는 각 플레이어에게 **개인화된 뷰**를 전송하여 상대방의 비공개 정보(손패, 뒤집어진 타일 등)가 노출되지 않도록 한다.

방 관리 이벤트

이벤트	방향	설명
`room:create`	C → S	방 생성 (이름, 게임 종류, 최대 인원)
`room:join`	C → S	대기 중인 방에 참가
`room:spectate`	C → S	게임 중인 방에 관전 입장
`room:leave`	C → S	방 퇴장
`room:update_settings`	C → S	방 설정 변경 (방장 전용)
`room:state`	S → Room	방 상태 브로드캐스트
`room:list`	S → Lobby	방 목록 갱신
`online:users`	S → Lobby	접속자 닉네임 목록

게임 공통 이벤트

이벤트	방향	설명
`game:start`	C → S	게임 시작 (방장, 2명 이상)
`game:state`	S → C	게임 상태 (플레이어별 개인화 뷰)
`game:aborted`	S → Room	게임 중단 (유저 퇴장)

젝스님트 전용 이벤트

이벤트	방향	설명
`game:select_card`	C → S	카드 선택
`game:unselect_card`	C → S	카드 선택 취소
`game:choose_row`	C → S	열 선택 (카드 < 모든 열)
`game:all_selected`	S → Room	전원 카드 공개
`game:event`	S → Room	카드 배치 / 열 회수
`game:round_end`	S → Room	라운드 점수
`game:next_round`	C → S	다음 라운드 준비 완료
`game:ready_status`	S → Room	준비 현황

다빈치 코드 전용 이벤트

이벤트	방향	설명
`davinci:draw`	C → S	타일 뽑기
`davinci:guess`	C → S	상대 타일 추측
`davinci:continue`	C → S	연속 추측
`davinci:stop`	C → S	턴 종료 (타일 비공개 배치)
`davinci:guess_result`	S → Room	추측 결과

채팅 이벤트

이벤트	방향	설명
`chat:send`	C → S	메시지 전송 (채널 지정)
`chat:message`	S → Channel	메시지 브로드캐스트
`chat:history`	양방향	최근 20개 메시지 요청/응답

7. 게임 로직 설계

7.1 젝스님트 (6 Nimmt!)

카드 범위는 1 ~ (인원수 x 10 + 4). 매 턴 모든 플레이어가 동시에 카드 1장을 선택하고, 낮은 카드부터 순서대로 4개 열에 배치한다.

게임 페이즈

Phase	설명
`selecting`	모든 플레이어가 동시에 카드 선택 (취소 가능, 마지막 1장 자동 제출)
`resolving`	낮은 카드부터 순서대로 열에 배치 (800ms 딜레이, 애니메이션)
`choosing_row`	카드가 모든 열보다 낮을 때 가져갈 열 선택
`round_end`	10턴 완료 후 점수 계산, 모든 플레이어 준비 시 다음 라운드
`game_over`	누적 66점 이상 도달 시 종료

벌점 규칙

조건	벌점	예시
55번 카드	7점	55
11의 배수	5점	11, 22, 33, 44, 66, 77, 88, 99
10의 배수	3점	10, 20, 30, 40, 50, 60, 70, 80, 90, 100
5의 배수	2점	5, 15, 25, 35, 45, 65, 75, 85, 95
나머지	1점	그 외 모든 카드

7.2 다빈치 코드 (Da Vinci Code)

검정/흰색 각 0~11 + 조커 = 총 26개 타일. 순차적으로 상대 타일을 추측하여 공개시키는 추리 게임이다.

게임 페이즈

Phase	설명
`drawing`	현재 플레이어가 타일 1개 뽑기 (풀 소진 시 생략)
`guessing`	상대 타일 1개 지목 + 숫자(또는 조커) 추측
`continue_or_stop`	정답 시: 연속 추측 또는 멈추기(타일 비공개 배치)
`game_over`	생존자 1명 남으면 종료

핵심 규칙

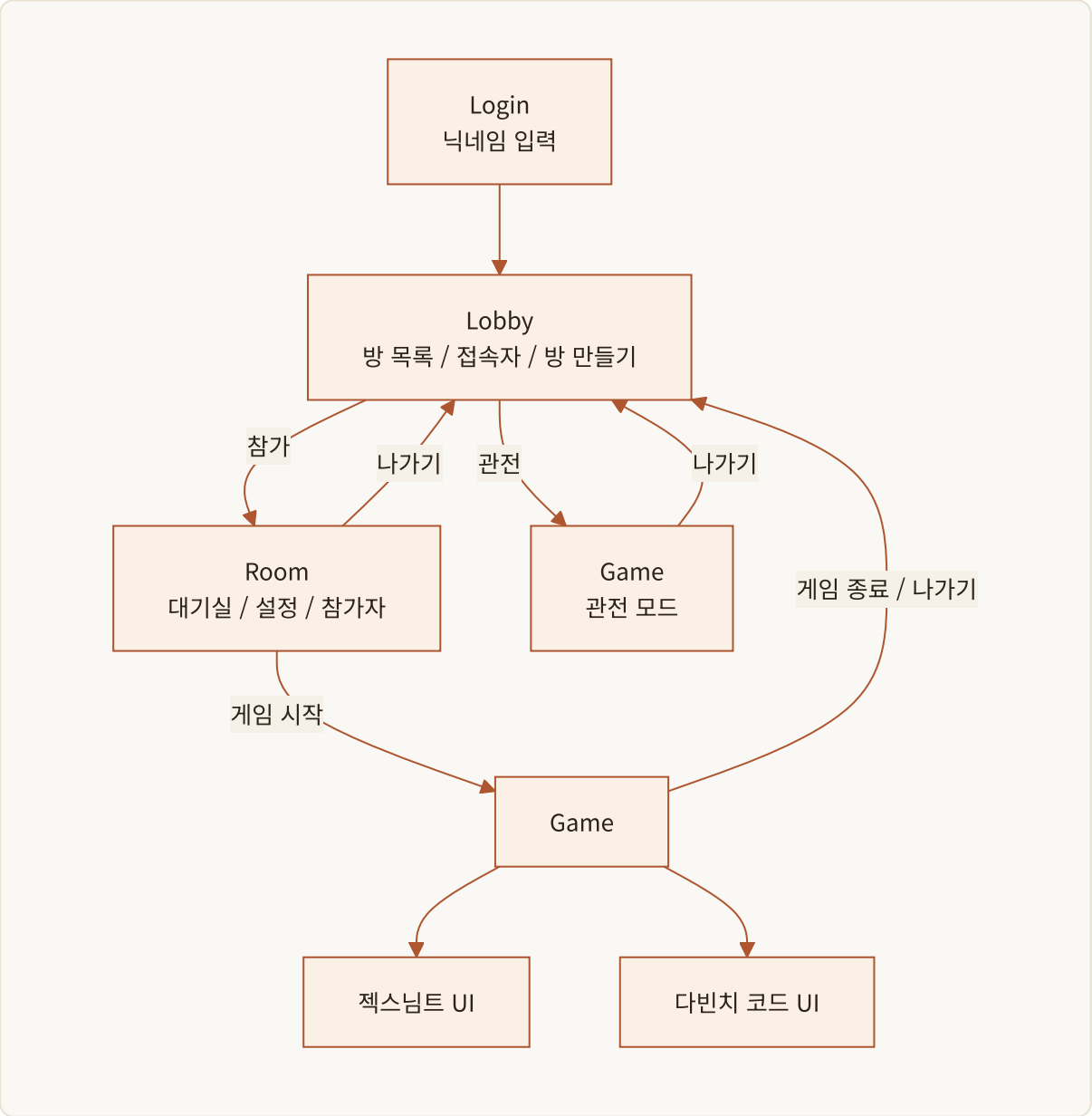
- **정렬**: 타일은 항상 왼쪽→오른쪽 오름차순, 같은 숫자면 검정이 왼쪽
- **조커**: 배치 시 자유 위치 선택 가능
- **오답 시**: 뽑은 타일을 공개 배치 (풀 소진 시 본인 타일 1개 공개)
- **탈락**: 모든 타일이 공개되면 탈락
- **승리**: 마지막 생존자

8. 클라이언트 구조

Context 계층

```
ThemeProvider          (테마 상태: ivory / excel)
  └─ AuthProvider      (토큰, 유저 정보, localStorage 연동)
      └─ SocketProvider  (Socket.IO 연결, JWT 인증)
          └─ BrowserRouter
              └─ AppRoutes  (인증 여부에 따른 라우팅)
```

페이지 흐름



게임 타입별 렌더링 분기

```
// Game.tsx 내부
if (gameState.gameType === 'davinci-code') {
  return <DaVinciGame socket={socket} gameState={gameState} />;
}
// 기본: 젝스님트 UI
```

채팅 시스템

환경	동작
PC (768px 이상)	우측 사이드바에 항상 표시 (300px)
모바일 (768px 이하)	FAB 버튼 클릭 시 우측에서 슬라이드 인

- 채널 기반: `lobby` 또는 `roomId`
- 입장 시 최근 20개 메시지 히스토리 로드
- 관전자 채팅: 닉네임 옆에 "(관전)" 표시

9. 스타일링 및 테마

CSS 변수 기반 테마 시스템

`data-theme` 속성으로 전환하며, `:root` 에 기본값(ivory), `[data-theme="excel"]` 에 오버라이드를 정의한다.

테마	특징
Ivory (기본)	아이보리 배경, 명조체, 둥근 모서리, 테라코타 액센트
Excel	회색 배경, 산세리프, 직각 모서리, 녹색 액센트, 줄무늬 테이블

반응형 설계 (768px 분기)

요소	PC	모바일
레이아웃	2컬럼 (본문 + 채팅)	1컬럼
채팅	우측 고정 사이드바	FAB + 슬라이드 패널
카드 크기	64 x 88px	48 x 66px
타일 크기	44 x 60px	36 x 50px
모달	고정 너비	화면 너비 적응

10. Docker 배포

서비스 구성

서비스	이미지	포트	역할
server	node:20-alpine	3001	API + Socket.IO
client	node:20-alpine	5173	Vite 개발 서버

개발 환경 특이사항

- **볼륨 마운트**: `./server/src` 와 `./client/src` 를 컨테이너에 마운트하여 소스 변경 시 자동 반영
- **폴링 모드**: Windows → Docker 환경에서 파일 변경 감지를 위해 Vite(`usePolling`)와 tsx watch(`CHOKIDAR_USEPOLLING`)에 폴링 활성화
- `index.html` 등 볼륨 외부 파일 변경 시에는 `docker compose up --build` 필요

실행 명령

```
# 시작 (빌드 포함)
docker compose up --build -d

# 종료
docker compose down

# Windows 배치 파일
start.bat / stop.bat
```

11. 서버 상태 관리

데이터베이스 없이 서버 프로세스 메모리에서 모든 상태를 관리한다.

저장소	타입	용도
`rooms`	Map<string, Room>	모든 활성 방 및 게임 상태
`userSockets`	Map<number, Socket>	유저 ID → 소켓 매핑
`onlineNicknames`	Set<string>	접속 중 닉네임 (중복 방지)
`chatHistory`	Map<string, Message[]>	채널별 최근 20개 메시지

> **주의:** 서버 재시작 시 모든 상태가 초기화된다. 영속성이 필요한 경우 데이터베이스 도입이 필요하다.

12. 보안 고려사항

- **JWT 시크릿**: 환경변수 `JWT_SECRET`으로 관리 (기본값은 개발 전용)
- **CORS**: 현재 와일드카드(`*`) 허용 — 프로덕션에서는 도메인 제한 필요
- **입력 검증**: 닉네임 20자 제한, 채팅 200자 제한
- **게임 상태 격리**: 플레이어별 뷰를 통해 상대방 비공개 정보 노출 방지
- **소켓 인증**: 모든 소켓 연결 시 JWT 필수 검증

13. 확장 가이드

새 게임 추가 시

단계	파일	작업
1	`server/src/games/{name}/logic.ts`	게임 로직 구현 (상태, 액션, 뷰 생성)
2	`server/src/socket/index.ts`	소켓 이벤트 핸들러 추가, game:start 분기
3	`client/src/components/{Name}Game.tsx`	게임 전용 UI 컴포넌트
4	`client/src/pages/Game.tsx`	`gameType` 분기 추가
5	`client/src/pages/Lobby.tsx`	게임 선택 옵션 추가
6	`client/src/styles/index.css`	게임별 스타일

새 테마 추가 시

단계	파일	작업
1	`client/src/contexts/ThemeContext.tsx`	`themes` 배열에 항목 추가
2	`client/src/styles/index.css`	`[data-theme="이름"]` 블록에 CSS 변수 정의